# VALOR: Vector-Aware Low-Rank Restructuring of Neural Networks for RISC-V Inference

**Zhihao Xu**[†]
School of Computer Science and Engineering
Southeast University
Nanjing, China
Hosea.xu@seu.edu.cn

**Xiaoning Du**[*]
Faculty of Information and Technology
Monash University
Melbourne, Australia
Xiaoning.du@monash.edu

**Bixin Li**[*]
School of Computer Science and Engineering
Southeast University
Nanjing, China
bx.li@seu.edu.cn

**Lulu Wang**
School of Computer Science and Engineering
Southeast University
Nanjing, China
wanglulu@seu.edu.cn

**Li Liao**
School of Computer Science and Engineering
Southeast University
Nanjing, China
lliao@seu.edu.cn

**Ying Zhou**
School of Computer Science and Engineering
Southeast University
Nanjing, China
zy@seu.edu.cn

## Abstract

The RISC-V Vector Extension (RVV) provides SIMD-style vector execution for accelerating deep learning (DL) inference on edge processors. However, existing low-rank compression methods mainly select ranks according to accuracy preservation and theoretical floating-point operation (FLOP) reduction, without considering whether the selected ranks are profitable under a target RVV effective vector length. As a result, a low-rank factorization that reduces FLOPs may still introduce tail-handling overhead or even increase the RVV instruction count under specific vector configurations. To address this problem, we propose VALOR, a low-rank restructuring framework for efficient DL deployment on RISC-V platforms. Given a pretrained model and a target RVV configuration, VALOR identifies decomposable layers, filters rank candidates using an RVV instruction-profitability condition, and searches for a global restructuring strategy with minimal accuracy loss. Furthermore, to avoid repeated end-to-end evaluation, VALOR uses a coupling-aware ranking proxy that combines normalized layer-wise factorization error with empirical-Fisher coupling estimates. Experiments on Spike and a real RISC-V vector processor, SpacemiT X60, show that VALOR achieves a better accuracy-efficiency trade-off than baseline methods.

## 1 Introduction

As a key branch of Machine Learning (ML), Deep Learning (DL) has experienced widespread adoption across various domains [15, 16] over the past decade. Increasingly, hardware and software vendors are deploying DL models on edge devices rather than cloud infrastructure to address security and privacy concerns [39, 24]. RISC-V has emerged as a promising open-source instruction set architecture for edge deployment thanks to its flexible customization options [33, 29]. This flexibility is particularly compelling for deep learning inference, where the RISC-V Vector Extension (RVV)

[*]Corresponding authors: Xiaoning Du and Bixin Li.
[†]Also affiliated with the Faculty of Information and Technology, Monash University, Melbourne, Australia. Email: Zhihao.xu@monash.edu.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

provides powerful Single Instruction Multiple Data (SIMD) capabilities to accelerate core tensor computations [17]. The key to RVV is the effective vector length, which determines how many elements can be processed by one vector instruction. These capabilities, resulting in architectural headroom, have catalyzed rapid ecosystem growth and industrial adoption. Currently, the RISC-V ecosystem has encompassed over 4,120 members across 52 countries, with more than 80 technical working groups driving standardization and toolchain maturity [38]. NVIDIA recently projected shipments of up to two billion RISC-V cores in a single year [38]. Therefore, developing efficient methodologies for deploying DL models on RISC-V processors has become increasingly critical to fully exploit the expanding hardware and software landscape.

Model compression is an important optimization strategy that aims to improve the execution efficiency of DL models on edge devices. Various compression methods, like pruning and quantization, have been studied and developed to model redundancy with different granularity. Among existing compression methods [26, 11, 10], *low-rank* compression is a unique method that can significantly reduce storage and computational resources. Based on the difference in the factorization strategy, these methods can be divided into 2-D matrix factorization and high-order tensor factorization. The key point of all factorization strategies is choosing an appropriate rank for factorization.

However, existing *low-rank* compression methods still fall short for efficient RISC-V deployment. On the one hand, conventional *low-rank* methods primarily optimize for theoretical floating-point operation (FLOP) reduction. They do not explicitly consider whether the selected rank matches the RVV's effective vector length, the most important characteristic of RISC-V hardware. If the selected rank is not divisible by the effective vector length, they may introduce scalar execution or tail handling, further increasing latency. We call these ranks non-instruction-profitable. As a result, a model factorized by non-instruction-profitable ranks with fewer FLOPs may still fail to translate its theoretical arithmetic reduction into latency reduction on real RISC-V processors. On the other hand, recent work such as HALOC [43] has explored hardware-aware *low-rank* compression by building hardware cost models from target-platform feedback, like latency and energy, and using them to guide automatic rank selection. However, this feedback-driven design is naturally tied to a specific hardware platform and execution profile for building its cost models. In RISC-V deployments, effective vector length may vary across implementations, precision modes and vector execution settings. Therefore, applying a HALOC-style method to RISC-V deployment would require rebuilding the hardware cost models and rerunning hardware-specific rank searches across different implementations, making hardware feedback-driven compression costly and introducing substantial engineering effort [43, 31]. Therefore, it is desirable to have a low-rank restructuring method that is aware of RVV effective vector lengths and can adapt pretrained models across different configurations.

To address this, we propose VALOR (Vector-Aware Low-Rank Restructuring), a low-rank restructuring method for efficient deployment of DL models on RISC-V platforms. Given a pretrained model and a target vector execution setting, VALOR first identifies decomposable layers and rewrites their weights through low-rank matrix factorization. Unlike conventional low-rank compression, VALOR restricts the rank to instruction-profitable candidates, enabling compression that reduces both arithmetic cost and RVV instruction overhead. VALOR then jointly optimizes rank choices across all selected layers to identify a restructuring strategy that balances accuracy preservation with reductions in FLOPs and instruction count. This global optimization is necessary because different layers have different sensitivity to low-rank approximation, and the cumulative accuracy loss is not simply the sum of independent layer-wise reconstruction errors [26, 27]. To guide this search without training from scratch, VALOR incorporates a heuristic low-cost ranking proxy motivated by a second-order analysis of the loss. By combining layer-wise reconstruction error with inter-layer coupling terms, the proxy predicts each low-rank restructuring strategy's impact on accuracy and enables efficient search over instruction-profitable rank configurations. Through this approach, VALOR unifies low-rank compression, vector-aware rank selection, and accuracy-aware global optimization into a coherent framework. It reduces theoretical FLOPs and parameters of DL models while improving instruction efficiency on RISC-V processors.

Experimental results on Spike and a real RISC-V processor demonstrate that VALOR provides a better accuracy-efficiency trade-off than the baseline. It achieves lower degradation in accuracy while reducing the instruction count and improving end-to-end latency. Furthermore, our method has been shown to generalize effectively across diverse DL models, including CNNs and hybrid models. In summary, this paper makes the following contributions:

- We present VALOR, a low-rank restructuring framework for efficient DL model deployment on RISC-V platforms. VALOR explicitly incorporates the RVV effective vector length into rank selection, enabling pretrained models to be adapted to different vector execution settings and avoiding the need to rebuild a measured latency cost model for every evaluated vector profile.
- We demonstrate that VALOR provides a better accuracy-efficiency trade-off than baseline methods. Experimental results on Spike and a real RISC-V processor show that VALOR achieves lower accuracy degradation while reducing the instruction count and improving end-to-end latency.
- We release our implementation as an open-source toolkit to facilitate reproducibility and further research in RISC-V-based edge AI deployment [3].

## 2 Background and Motivation

In this section, we briefly introduce the SIMD and RVV extensions of the RISC-V architecture and explain their key features. Then, we use an example to motivate our methodology.

### 2.1 SIMD and RVV

Conventional CPU instructions follow the Single Instruction Single Data (SISD) principle, in which each instruction processes a single data element. In contrast, SIMD accelerates computation by enabling one instruction to process multiple data elements simultaneously. The RVV Extension is a vector-processing extension for the RISC-V architecture [2, 32], providing powerful SIMD capabilities for efficient parallel computation in high-performance computing and deep learning inference. In RVV, the number of elements processed by one vector instruction is determined by the hardware vector register length $VLEN$, the selected element width $SEW$, and the register grouping factor $LMUL$. For a given vector execution setting, the maximum number of elements that can be processed by a vector instruction is referred to as the effective vector length, denoted as $\eta$, where $\eta = \frac{VLEN}{SEW} \times LMUL$. RVV programs set the runtime vector length through vector configuration instructions. In regular vectorized loops, full iterations typically execute with $\eta$. If low-rank compression introduces a rank that becomes a new matrix dimension and the rank is not divisible by $\eta$, the remaining part may require tail handling, which sets a smaller runtime vector length than $\eta$ to handle the remaining part. But tail handling may increase instruction overhead and reduce vector efficiency. We examine this challenge in the following section.

### 2.2 Motivation

Consider a matrix multiplication $Y = XW$, where $X \in \mathbb{R}^{B\times K}$ and $W \in \mathbb{R}^{K\times N}$. A conventional low-rank compression method decomposes the weight matrix into two smaller matrices, $W \approx UV$, where $U \in \mathbb{R}^{K\times r}$ and $V \in \mathbb{R}^{r\times N}$. The original multiplication is then rewritten as $Y \approx (XU)V$. The rank $r$ is mathematically valid when $1 \le r \le \min(K, N)$. In terms of FLOPs, the original multiplication requires $F_{\text{orig}} = 2BKN$, while the factorized multiplication requires $F_{\text{lr}}(r) = 2BKr + 2BrN = 2Br(K + N)$. Therefore, a rank is FLOP profitable when $F_{\text{lr}}(r) < F_{\text{orig}}$. However, FLOP profitability does not guarantee instruction profitability on RVV processors. Let $\eta$ be the effective vector length. Following the instruction specification of RVV, each output element of a matrix multiplication is computed as a strip-mined dot product along the reduction dimension. For the original multiplication $XW$, each output element accumulates over $K$ elements and there are $N$ output elements for each input row. Therefore, for each batch element, the full-vector part contributes $N\lfloor K/\eta \rfloor (I_{\text{vec}} + I_{\text{cfg}})$ instructions, where $I_{\text{vec}}$ denotes the instruction count of one full-vector computation step, and $I_{\text{cfg}}$ denotes the associated vector-configuration overhead. If $K \bmod \eta \neq 0$, the remaining elements require a tail cleanup, contributing an additional $NI_{\text{tail}}$ instructions, where $I_{\text{tail}}$ denotes the instruction count of one tail-cleanup step. Thus, the original multiplication has instruction count $I_{\text{orig}} = BN(\lfloor K/\eta \rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [K \bmod \eta \neq 0]I_{\text{tail}})$, where $[\cdot]$ is the indicator function, which equals 1 when the condition holds and 0 otherwise. After low-rank factorization, the first multiplication $XU$ has reduction dimension $K$ and output width $r$, while the second multiplication $(XU)V$ has reduction dimension $r$ and output width $N$. Therefore, the factorized multiplication has instruction count $I_{\text{lr}}(r) = Br(\lfloor K/\eta \rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [K \bmod \eta \neq 0]I_{\text{tail}}) + BN(\lfloor r/\eta \rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [r \bmod \eta \neq 0]I_{\text{tail}})$.

Now consider a concrete example with $B = 128$, $K = 512$, $N = 1024$, and $\eta = 256$. If a conventional low-rank method selects rank as $r = 257$, then $F_{\text{lr}}(257)/F_{\text{orig}} = 257(512 + 1024)/(512 \times$

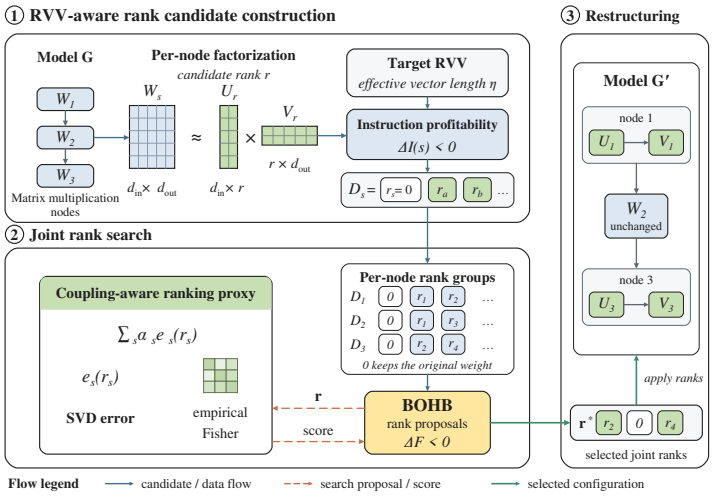



Figure 1: The workflow of VALOR.

$1024) = 75.3\%$, so this rank appears effective from the perspective of FLOP reduction. However, $r = 257$ has become the reduction dimension of the second multiplication. Since the rank is not divisible by $\eta = 256$, each output element requires one full-vector iteration and one tail operation. The tail part alone contributes $128 \times 1024 \times I_{\text{tail}}$ additional instructions. In practice, tail handling is typically implemented as a single vector iteration guarded by an adjusted vector length with fixed configuration instructions [35, 34, 1, 4], so it can be approximated as $I_{\text{tail}} = I_{\text{vec}} + I_{\text{cfg}}$. Under this approximation, the factorized multiplication requires $I_{\text{lr}} = 128(257 \times 2I_{\text{tail}} + 1024 \times 2I_{\text{tail}}) = 327936I_{\text{tail}}$, whereas the original multiplication requires $I_{\text{orig}} = 128 \times 1024 \times 2I_{\text{tail}} = 262144I_{\text{tail}}$. Therefore, although the rank reduces FLOPs, it increases the RVV instruction count.

## 3 Framework

This section presents VALOR, a framework that automatically optimizes DL models for efficient deployment on RISC-V vector processors. We first formalize the tail operation instruction reduction problem, then describe our factorization strategy and optimization process.

### 3.1 Overview

The framework of VALOR is shown in Figure 1. Given a pretrained model and a target RVV execution setting, VALOR first identifies layers that can be restructured using low-rank factorization, namely matrix multiplication nodes. For each matrix multiplication node, VALOR builds a rank candidate group by choosing valid ranks that are instruction-profitable under the target effective vector length. But unlike the FLOPs, which can be computed directly from model dimensions, building an exact instruction model for the entire neural network is difficult. Because different rank choices may change many lowered kernels and their generated instruction sequences. VALOR addresses this challenge by modeling instruction-profitable conditions at the node level. This modeling converts RVV-dependent instruction behavior into a rank candidate group for each node. The candidate groups across all nodes form a discrete global search space. VALOR searches this space to select ranks that minimize the proxy-ranked accuracy degradation while satisfying instruction-count and FLOP-reduction constraints. To avoid repeated end-to-end evaluation, VALOR uses a Hessian-aware analytical predictor that combines local factorization error with inter-layer coupling. The output is a restructured model that improves execution efficiency while minimizing the accuracy loss.

### 3.2 Problem Formulation

We define a general neural network as $G = (V, E)$, a finite, acyclic computation graph. Each node $v \in V$ computes a single-operation function

$$\varphi_v : \mathcal{Z}^{\text{in}(v)} \otimes \Theta_v \to \mathcal{Z},$$

where $\mathcal{Z}$ is a finite-dimensional real vector space (e.g. $\mathbb{R}^d$ or a product of such spaces), $\mathrm{in}(v)$ is the set of incoming edges to $v$, $\Theta_v$ is the parameter space for node $v$, and $\otimes$ represents typical calculus operators in neural networks, such as dot product, addition, and convolution.

Given parameters $\theta = \{\theta_v\}_{v \in V} \in \Theta \colon \prod_v \Theta_v$, the model $f_\theta : \mathcal{X} \to \mathcal{Y}$ is defined by evaluating the graph in topological order, feeding an input $x \in \mathcal{X} \subseteq \mathcal{Z}$ at the designated input node(s), and reading the value at the designated output node(s) as $f_\theta(x) \in \mathcal{Y} \subseteq \mathcal{Z}$.

Identify the set $S \subseteq V$ of linear parameterized nodes, i.e., nodes whose learnable action can be represented as, or lowered to, right-multiplication by a matrix, and we refer to them as **matrix-multiplication** nodes, which are decomposable layers. For $s \in S$, with dimensions $d_{\text{in}}(s), d_{\text{out}}(s) \in \mathbb{N}$, its parameter is denoted as:

$$W_s \in \mathbb{R}^{\,d_{\text{in}}(s) \times d_{\text{out}}(s)},$$

Thus, $s$ is a linear map

$$T^{(s)}_{W_s} : \mathbb{R}^{d_{\text{in}}(s)} \to \mathbb{R}^{d_{\text{out}}(s)}, \qquad T^{(s)}_{W_s}(z) = zW_s.$$

For each selected node $s \in S$, VALOR restructures its weight matrix $W_s \in \mathbb{R}^{d_{\text{in}}(s) \times d_{\text{out}}(s)}$ through a low-rank factorization. Given a rank $r_s$, node $s$ is replaced by two consecutive matrix-multiplication nodes with parameters

$$W_u \in \mathbb{R}^{d_{\text{in}}(s) \times r_s}, \qquad W_r \in \mathbb{R}^{r_s \times d_{\text{out}}(s)},$$

such that

$$W_s \;\approx\; W_u\, W_r.$$

For an input activation $z$ to node $s$, the restructured computation is

$$T^{(s)}_{W_s}(z) \;=\; zW_s \;\approx\; z(W_u W_r) \;=\; (zW_u)W_r \;=\; \big(T^{(r)}_{W_r} \circ T^{(u)}_{W_u}\big)(z),$$

Thus, the transformation replaces one parameterized linear node with two smaller linear nodes while approximately preserving the original computation. By factorizing $s$ to $u$ and $r$, the computation graph $G = (V, E)$ will be transformed to $G' = (V', E')$ by

$$V' \;=\; (V \setminus \{s\}) \cup \{u, r\},$$

$$E' \;=\; \big(E \setminus \{(i,s), (s,j) : (i,s) \in E \text{ or } (s,j) \in E\}\big) \cup$$
$$\{(i,u) :\; (i,s) \in E\} \;\cup\; \{(u,r)\} \;\cup\; \{(r,j) :\; (s,j) \in E\}.$$

That is, all incoming edges to $s$ now feed $u$, the output of $u$ feeds $r$, and all outgoing edges from $s$ now originate at $r$. The factorization of all sites in $S$ can be carried out independently, with the factorization applied to each site only once. We denote the resulting graph $G'$ as being associated with the parameter $\theta'$. Here we note the accuracy loss over an input set $D$ as:

$$\Delta\mathcal{L}_D(\theta', \theta) = \frac{1}{|D|} \sum_{x \in D} d\big(f_{\theta'}(x), f_\theta(x)\big),$$

Furthermore, for node $s$, the rank $r_s$ must first satisfy the valid-rank constraint $1 \le r_s \le \min\big(d_{\text{in}}(s), d_{\text{out}}(s)\big)$. But this constraint only ensures that $r_s$ is a valid matrix rank. It does not guarantee that the restructured node reduces the execution time on the target RVV processor. Therefore, we need to filter the ranks based on a decreasing execution time condition. Estimating execution time without actually running the code is quite challenging. Instead, we analyze the counts of different instruction types and conduct a comparative analysis. Generally, for a site $s$ and the effective vector length $\eta$, the total instruction count before factorization can be denoted as:

$$I(G; s) \;=\; d_{\text{out}}(s) \left\lfloor \frac{d_{\text{in}}(s)}{\eta} \right\rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [d_{\text{in}}(s) \bmod \eta] \cdot I_{\text{tail}}$$

Then the total instruction count after factorization can be denoted as:

$$I(G'; s) \;=\; r_s \left\lfloor \frac{d_{\text{in}}(s)}{\eta} \right\rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [d_{\text{in}}(s) \bmod \eta] \cdot I_{\text{tail}}$$
$$+ d_{\text{out}}(s) \left\lfloor \frac{r_s}{\eta} \right\rfloor (I_{\text{vec}} + I_{\text{cfg}}) + [r_s \bmod \eta] \cdot I_{\text{tail}}$$

Hence, a rank is instruction-profitable if:

$$\triangle I(s) = I(G'; s) - I(G; s) < 0,$$

**Approximation.** According to prior work on RVV vectorization [35, 34, 1, 4], tail handling in RVV architectures is typically implemented as a single vector iteration guarded by an adjusted vector length with fixed configuration instructions. Consequently, the tail overhead can be approximated as the vector computation instruction plus the configuration instruction: $I_{\text{tail}} = I_{\text{vec}} + I_{\text{cfg}}$. Under this assumption, the instruction-profitable condition of rank can be denoted as:

$$r_s < d_{\text{out}}(s) \left(1 - \frac{\lceil r_s/\eta \rceil}{\lceil d_{\text{in}}(s)/\eta \rceil}\right)$$

**Scaling.** According to the ceiling upbound $\left\lceil \frac{r}{V} \right\rceil \leq \frac{r}{V} + 1$.We can obtain a sufficient (but potentially conservative) instruction-profitable condition of rank, which can be denoted as:

$$r_s < \frac{d_{\text{out}}(s) \left( \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil - 1 \right)}{\left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil + \frac{d_{\text{out}}(s)}{\eta}}$$

We will prove these conditions in Appendix A and validate the approximation in Appendix E.1.

**Our goal.** The goal of VALOR is to address a single optimization problem over input set $D$, which is to find a transformed graph $G'$ that minimizes the accuracy degradation while reducing the FLOP $F$ and instruction counts:

$$\min_{\{r_s\}_{s \in S}} \quad \Delta\mathcal{L}_D(\theta', \theta) \text{s.t.} \qquad \triangle I(s) < 0, \triangle F < 0.$$

To examine a different deployment preference, Appendix B further presents an accuracy-tolerance formulation that maximizes efficiency gains subject to a bounded accuracy loss.

### 3.3 Optimization Problem Solving

To obtain a transformed graph $G'$ that minimizes the accuracy loss while reducing instruction count and FLOPs, VALOR formulates a discrete, high-dimensional search over low-rank restructuring strategies. For each matrix-multiplication node $s \in S$, the viable restructuring choices form a candidate group:

$$D_s = \{0\} \cup \{\, r_s \in \mathbb{N} \mid 1 \leq r_s \leq \min(d_{\text{in}}(s), d_{\text{out}}(s)),\ \Delta I(s) < 0 \},$$

where $r_s = 0$ means that node $s$ is kept unchanged, and $r_s > 0$ specifies the rank used to factorize node $s$. The candidate groups across all nodes define the global strategy space, and a restructuring strategy is represented by

$$\mathbf{r} = \{r_s\}_{s \in S} \in \prod_{s \in S} D_s.$$

Given a rank assignment $\{r_s\}_{s \in S}$, VALOR applies low-rank factorization only to nodes with $r_s > 0$, yielding the transformed graph $G'$ with parameter $\theta'$. Direct end-to-end evaluation of each $\mathbf{r}$ is prohibitively expensive. VALOR therefore adopts Bayesian Optimization and Hyperband (BOHB) [13] to explore the candidate strategy space, using low-cost proxy evaluations for rapid screening and a small number of held-out full assessments to confirm the final candidate strategy. During search, candidates that do not satisfy the FLOP-reduction constraint $\triangle F < 0$ are filtered out. Among the remaining candidates, VALOR ranks the strategies according to the proxy score $\widehat{\Delta\mathcal{L}}(\mathbf{r})$ and selects the one with the smallest score. We report the BOHB search budget, a budget-matched random-search comparison and wall-clock search time in Appendix E.2.

As a low-cost heuristic proxy, VALOR uses a coupling-aware ranking proxy motivated by the Hessian-based second-order approximation of the loss change $\widehat{\Delta\mathcal{L}}(\mathbf{r})$ to screen candidate strategies before without repeated end-to-end evaluation. Let $S_{\mathbf{r}} = \{s \in S \mid r_s > 0\}$ denote the set of transformed nodes under strategy $\mathbf{r}$. For each transformed node $s \in S_{\mathbf{r}}$, the factors $W_u$ and $W_r$ are obtained by truncated SVD of $W_s$ under the selected rank $r_s$. The local distortion introduced at node $s$ is measured by the normalized factorization error:

$$e_s(r_s) = \ell_s = \frac{\|W_s - W_u W_r\|_F^2}{\|W_s\|_F^2}.$$

This term captures how much the selected rank changes the local linear operator, but it does not capture how this local distortion affects the global model loss. Different layers can have different loss sensitivity, and distortions introduced at earlier nodes may also affect later transformed nodes. To account for this effect, VALOR estimates the global accuracy degradation by weighting the local distortion of each transformed node:

$$\widehat{\Delta\mathcal{L}}(\mathbf{r}) = \sum_{s \in S_{\mathbf{r}}} \alpha_s \cdot e_s(r_s).$$

The coefficient $\alpha_s \geq 1$ heuristically models the amplification of the local distortion at node $s$. We motivate it with a second-order expansion of the loss in vectorized parameter perturbations, where diagonal Hessian blocks capture layer-wise curvature and off-diagonal blocks capture cross-layer coupling. It's a lightweight heuristic to account for potential cross-layer interactions without explicitly evaluating the full Hessian-based loss change. Specifically, we define

$$\alpha_s = 1 + \sum_{t \prec s,\ t \in S_{\mathbf{r}}} \ell_t \cdot g_{ts},$$

where $t \prec s$ denotes a transformed node that precedes $s$ in the topological order, and $\ell_t$ is the local factorization error introduced at node $t$. Let $H$ denote the Hessian of the loss on the input set $D$ with respect to the model parameters, and let $H_{ts}$ denote the Hessian block that couples the parameters of nodes $t$ and $s$. We define the normalized coupling score as

$$g_{ts} = \frac{\|H_{ts}\|_F}{\sqrt{\|H_{tt}\|_F \cdot \|H_{ss}\|_F}}.$$

The normalization makes $g_{ts}$ dimensionless and comparable across node pairs. Since the Frobenius-norm normalization does not guarantee a strict upper bound in general, we clip the normalized score to $[0, 1]$ in implementation. A larger $g_{ts}$ indicates stronger estimated loss-curvature coupling between two transformed nodes. VALOR does not explicitly construct the full Hessian. Instead, it estimates the required normalized coupling scores on a small subset of $D$ using a block-wise empirical Fisher approximation, which uses gradients of the loss with respect to the parameters of the corresponding nodes. We provide the vectorized derivation, state the approximation boundaries, and compare this proxy with simpler alternatives in Appendix C and Appendix E.

# 4 Experiments

To validate the effectiveness of VALOR, we conduct extensive experiments across various DL models. We also compare VALOR against baseline methods [26, 11, 10, 21]. Our experiments aim to answer the following research questions:

- *RQ1: How does VALOR improve vector instruction utilization and execution efficiency compared with existing methods?*
- *RQ2: Does VALOR generalize to real-world RISC-V hardware with practical speedup?*
- *RQ3: How does each component of VALOR contribute to the overall performance gains?*

## 4.1 Experimental settings

**Datasets and Models.** We include four representative model–dataset pairs covering diverse architectures and classification tasks: (i) ResNet-50 [15] on ImageNet-1k [8] for large-scale image classification; (ii) MobileNet [37] on ImageNet-1k [8] for efficient classification on resource-constrained devices; (iii) ArcFace [9] with a LeResNet backbone on Labeled Faces in the Wild (LFW) [19] for face verification; and (iv) YOLOv4 [6] on COCO 2017 dataset [28] for object detection.

**Implementation Details.** We adopt the standard TVM [7] deployment pipeline. Pretrained models are imported into Relay [7, 36], optimized, and restructured by VALOR before compilation with the RISC-V RVV backend. To evaluate the impact of RVV profile diversity, we use multiple vector configurations on Spike by varying `VLEN`$\in \{128, 256\}$ and `LMUL`$\in \{1, 2, 4, 8\}$ under different element-width settings with `SEW`$\in \{8, 16, 32\}$. These configurations correspond to seven effective vector lengths, $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$, where $\eta = (\texttt{VLEN}/\texttt{SEW}) \times \texttt{LMUL}$. We use tail-agnostic and mask-agnostic policies for all runs. Baselines are compiled with the same TVM pipeline and

Table 1: Overall effectiveness of VALOR across models and RVV configurations. Instruction reduction is averaged over $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$.

| | ResNet-50 | | | MobileNetV2 | | | ArcFace | | | YOLOv4 | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ |
| SVD | 1.31 | 37.6 | 33.1 | 1.95 | 31.7 | 27.4 | 0.41 | 35.9 | 31.7 | 1.87 | 33.8 | 29.5 |
| ALDS | 1.04 | 40.3 | 36.2 | 1.47 | 34.5 | 30.9 | 0.34 | 38.5 | 34.3 | 1.44 | 36.4 | 32.7 |
| BSR | 1.62 | 34.8 | 30.5 | 2.23 | 28.9 | 24.6 | 0.49 | 33.2 | 29.1 | 2.26 | 31.2 | 26.8 |
| LCNN | 1.48 | 36.1 | 31.6 | 2.11 | 30.2 | 25.8 | 0.45 | 34.5 | 30.2 | 2.05 | 32.5 | 28.1 |
| **VALOR** | **0.75** | **43.1** | **41.8** | **0.96** | **36.8** | **35.3** | **0.25** | **41.6** | **39.8** | **0.97** | **39.7** | **37.4** |

A.L. = Accuracy Loss; F.R. = FLOPs Reduction (%); I.R. = Instruction Reduction (%).

evaluated under the same RVV settings to ensure fairness. After rank selection, all compressed models, including VALOR and all baselines, are fine-tuned with the same training budget and hyperparameter settings. All results are reported as the mean over three runs. In implementation, we adopt the tail-cost approximation in Section 3. This approximation enables VALOR to evaluate more rank choices across different effective vector lengths and expose the algorithmic benefits of RVV-aware restructuring. In RQ2, we further evaluate two variants of VALOR on a real RISC-V vector processor, SpacemiT X60 with `VLEN=128`. The first variant uses the approximation-based condition, while the second uses the scaling sufficient condition. This comparison evaluates the practical impact of the approximation and examines how conservative the scaling bound is on real hardware.

**Baseline methods.** We compare VALOR with four low-rank compression baselines [10, 26, 11, 21]. First, **SVD** [10] applies standard post-training low-rank decomposition to matrix-multiplication nodes and selects ranks without considering RVV execution behavior. Second, **ALDS** [26] performs automatic layer-wise rank allocation to obtain a global low-rank compression strategy, but its rank selection is hardware-agnostic and does not model RVV instruction profitability. Third, **BSR** [11] uses beam-search-based joint rank selection to optimize layer-wise ranks. Fourth, **LCNN** [21] learns layer-wise ranks for low-rank neural network compression by optimizing compression and accuracy objectives, but it does not account for RVV-specific instruction behavior. All baselines are compiled with the same TVM-RVV toolchain and evaluated under the same RVV configurations as VALOR. In addition, we include a **HALOC-style** hardware-feedback baseline [43] in RQ2 on SpacemiT X60 to compare VALOR with a feedback-driven rank-selection strategy under a single real RVV setting.

**Metrics.** We evaluate VALOR using three primary metrics: task accuracy, instruction-count reduction, and FLOP reduction. For task accuracy, we report the community-standard metric on each benchmark dataset, including top-1 accuracy, verification accuracy, and mAP. Instruction-count reduction measures the RVV-level execution-cost decrease under each target vector configuration, while FLOP reduction measures the theoretical arithmetic-cost decrease introduced by low-rank restructuring. For real-world RISC-V hardware, we additionally report end-to-end latency to validate whether instruction-count reduction translates into practical speedup.

### 4.2 RQ1: Execution Efficiency Compared to Baselines

Table 1 compares VALOR with four low-rank baselines, including SVD, ALDS, BSR, and LCNN, across four representative model–dataset pairs. The reported results of all methods are averaged over seven effective vector lengths, $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$. Detailed per-$\eta$ results are reported in Appendix D. VALOR consistently achieves the best overall trade-off. Compared with the strongest baseline, ALDS, VALOR reduces the average accuracy loss from $1.07$ to $0.73$, while increasing the average FLOP reduction from $37.4\%$ to $40.3\%$ and the average instruction reduction from $33.5\%$ to $38.6\%$. Across the four models, VALOR improves instruction reduction over ALDS by $4.4$ to $5.6$ percentage points while also achieving lower accuracy loss. Compared with SVD, BSR, and LCNN, VALOR also consistently provides higher FLOP reduction and higher instruction reduction. These results show that RVV-aware rank selection improves both arithmetic reduction and vector-level execution efficiency under the same settings.

Table 2: Relative reductions in latency and accuracy on SpacemiT X60. VALOR is evaluated under both Approximation-based and Scaling-based policies.

| | ResNet-50 | | MobileNetV2 | | ArcFace | | YOLOv4 | |
|---|---|---|---|---|---|---|---|---|
| **Method** | ΔLat.↑ | ΔAcc.↓ | ΔLat.↑ | ΔAcc.↓ | ΔLat.↑ | ΔAcc.↓ | ΔLat.↑ | ΔAcc.↓ |
| SVD | 25.8 | 1.31 | 21.4 | 1.95 | 24.7 | 0.41 | 23.0 | 1.87 |
| ALDS | 29.0 | 1.04 | 24.7 | 1.47 | 27.4 | 0.34 | 26.2 | 1.44 |
| BSR | 23.2 | 1.62 | 18.7 | 2.23 | 22.1 | 0.49 | 20.4 | 2.26 |
| LCNN | 24.5 | 1.48 | 19.8 | 2.11 | 23.3 | 0.45 | 21.6 | 2.05 |
| HALOC-style | 37.4 | 0.81 | 30.6 | 1.02 | 34.5 | 0.28 | 32.1 | 1.04 |
| $\text{VALOR}_{\text{approx}}$ | **38.2** | **0.75** | **31.4** | **0.96** | **35.1** | **0.25** | **33.0** | **0.97** |
| $\text{VALOR}_{\text{scale}}$ | **38.2** | **0.75** | 28.7 | 1.13 | 27.9 | 0.31 | **33.0** | **0.97** |

All values in %.

ΔLat. = Latency reduction; ΔAcc. = Accuracy loss.

### 4.3 RQ2: Generalization to Real-world RVV Hardware

Table 2 reports the latency reduction and accuracy loss on SpacemiT X60. As a commercially available RISC-V processor with RVV support [30, 5], SpacemiT X60 allows us to validate whether the instruction-count reduction predicted by VALOR translates into practical on-device speedup. All values are measured relative to the original implementation without low-rank restructuring. Overall, $\text{VALOR}_{\text{approx}}$ achieves the largest latency reductions and the lowest accuracy loss across all four models. Compared with ALDS, the strongest hardware-agnostic baseline in most cases, $\text{VALOR}_{\text{approx}}$ improves latency reduction from $29.0\%$ to $38.2\%$ on ResNet-50, from $24.7\%$ to $31.4\%$ on MobileNetV2, from $27.4\%$ to $35.1\%$ on ArcFace, and from $26.2\%$ to $33.0\%$ on YOLOv4. Compared with the HALOC-style feedback baseline, the observed differences are small. We therefore interpret the two methods as providing comparable performance in this evaluated setting rather than claim a decisive latency advantage. VALOR's practical distinction is that its RVV-parameterized profitability model does not require constructing a separate measured latency cost model for each vector profile. At the same time, $\text{VALOR}_{\text{approx}}$ keeps the accuracy loss below $1\%$ across all four models, while most baselines incur larger accuracy loss on multiple models. These results show that VALOR provides practical speedups on real RISC-V vector hardware while preserving model accuracy. Comparing the two VALOR variants, $\text{VALOR}_{\text{approx}}$ generally achieves better latency reduction than $\text{VALOR}_{\text{scale}}$. On ResNet-50 and YOLOv4, both variants produce identical results, indicating that the scaling sufficient condition preserves the same effective restructuring choices. On MobileNetV2 and ArcFace, $\text{VALOR}_{\text{scale}}$ is more conservative and rejects some ranks that remain profitable in practice. For example, on ArcFace, $\text{VALOR}_{\text{approx}}$ reduces latency by $35.1\%$, while $\text{VALOR}_{\text{scale}}$ reduces latency by $27.9\%$. These results suggest that the approximation-based policy exposes more profitable restructuring choices on the evaluated hardware.

### 4.4 RQ3: Ablation Studies

We compare VALOR with the low-cost proxy evaluations (Section 3.3) enabled (LCP-On) and disabled (LCP-Off). Both settings use the same toolchain and RVV profile. Table 3 isolates the candidate-ranking stage and compares proxy-guided ranking (LCP-On) with full-measurement ranking (LCP-Off). Across all four workloads, LCP-On reaches the same reported instruction reductions and task accuracies as LCP-Off while requiring approximately $40\%$ of its average ranking-stage time. The complete BOHB search, which also includes candidate construction and held-out full validations, is reported separately in Appendix E.2.

Table 3: Candidate-ranking-stage ablation on time and accuracy.

| **Model** | **Method** | **I.R.↑** | **Acc.↑** | **Time↓** |
|---|---|---|---|---|
| ResNet-50 | On | 41.8 | 75.38 | 140 |
| | Off | 41.8 | 75.38 | 430 |
| MobileNetV2 | On | 35.3 | 70.92 | 170 |
| | Off | 35.3 | 70.92 | 480 |
| ArcFace | On | 39.8 | 99.57 | 270 |
| | Off | 39.8 | 99.57 | 620 |
| YOLOv4 | On | 37.4 | 40.25 | 300 |
| | Off | 37.4 | 40.25 | 720 |

I.R. = Instruction Reduction (%); Acc. = Task Accuracy (%); Time in seconds.

## 5 Related Work

Achieving efficient acceleration of AI model reasoning on the RISC-V platform, which can be divided into two levels: the software level and the hardware level.

**Software-level efficiency.** Model compression reduces inference cost through knowledge distillation [14, 18], pruning [12, 42, 40], quantization [20, 22], and low-rank decomposition [10]. Low-rank decomposition is particularly relevant because it restructures matrix multiplication and convolutional layers into smaller factorized operators. Recent methods such as ALDS [26] and BSR [11] further improve layer-wise rank allocation. However, these methods mainly optimize compression and accuracy, without explicitly checking whether selected ranks are instruction-profitable under a specific RVV effective vector length. Recently, hardware-aware low-rank methods, such as HALOC [43], incorporate latency or hardware-cost feedback into rank search. These approaches are effective when training-time search, hardware feedback, and fine-tuning are available. VALOR targets a different deployment setting, where pretrained models can be adapted to diverse RISC-V platforms.

**Hardware Level.** At the hardware level, many works focus on the design of customized instructions to enable efficient AI model reasoning. For example, Titopoulos et al. [41] proposed a new customized vector instruction to accelerate sparse neural network matrix multiplication. Jeong et al. present VEGETA [23], a set of ISA and microarchitecture extensions over dense matrix engines to support various DL models with different sparsity levels. Similarly, Li et al. [25] designed a CNN processor based on the RISC-V core, featuring custom instructions to accelerate convolution processing. However, these methods primarily reduce parameter counts and do not resolve the mismatch between neural network dimensions and the RVV vector length.

## 6 Conclusion and Limitation

We presented VALOR, a post-training model restructuring framework for efficient deep learning inference on RISC-V processors with RVV support. VALOR applies low-rank factorization to matrix-multiplication nodes and selects ranks that are instruction-profitable under the target effective vector length, reducing both FLOPs and RVV instruction cost without training from scratch. Experimental results across four models and seven simulated RVV profiles show that VALOR achieves lower accuracy loss and higher instruction reduction than baseline methods, and evaluation on SpacemiT X60 confirms practical latency benefits under the evaluated TVM–RVV pipeline.

One limitation of our evaluation is that we use a fixed TVM-based compilation pipeline and only one physical RVV processor. Backend code generation, operator fusion, memory layout, cache behavior, register pressure, instruction scheduling, and vectorization schedules can affect final latency, so the analytical instruction-count model does not capture every processor- and compiler-dependent effect. The current evidence therefore supports portability across the seven evaluated RVV profiles within this pipeline, not latency optimality across arbitrary RISC-V processors or other SIMD ISAs. Extending VALOR to another backend would require replacing the RVV-specific profitability model with a backend-specific vector–scalar cost model; the global rank-search framework and ranking proxy could remain unchanged. Future work will evaluate more compilers and processors and study lightweight fine-tuning strategies after restructuring.

## Acknowledgments and Disclosure of Funding

This work is jointly supported by the Key Research and Development Program of Jiangsu Province (No. BE2021002-3) and the National Key Research and Development Program of China (No. 2019YFE0105500).

## References

[1] Neil Adit and Adrian Sampson. Performance left on the table: An evaluation of compiler autovectorization for risc-v. *IEEE Micro*, 42(5):41–48, 2022.

[2] Alon Amid, Krste Asanovic, Allen Baum, Alex Bradbury, Tony Brewer, Chris Celio, Aliaksei Chapyzhenka, Silviu Chiricescu, Ken Dockser, Bob Dreyer, Roger Espasa, Sean Halle, John Hauser, David Horner, Bruce Hoult, Bill Huffman, Nicholas Knight, Constantine Korikov, Ben Korpan, Hanna Kruppe, Yunsup Lee, Guy

Lemieux, Grigorios Magklis, Filip Moc, Rich Newell, Albert Ou, David Patterson, Colin Schmidt, Alex Solomatnikov, Steve Wallach, Andrew Waterman, and Jim Wilson. The risc-v base vector extension, version 1.0. `https://lists.riscv.org/g/tech-vector-ext/attachment/686/0/riscv-v-spec.pdf`, Sep 2021. Accessed: 2025-09-16.

[3] Anonymous Authors. VALOR: Vector-aware low-rank restructuring of neural networks for risc-v inference. `https://anonymous.4open.science/r/VALOR-428C/`, 2026. Anonymous artifact repository.

[4] Adrià Armejach, Helena Caminal, Juan M Cebrian, Rekai González-Alberquilla, Chris Adeniyi-Jones, Mateo Valero, Marc Casas, and Miquel Moretó. Stencil codes on a vector length agnostic architecture. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, pages 1–12, 2018.

[5] Alexander Batashev. Dissecting risc-v performance: Practical pmu profiling and hardware-agnostic roofline analysis on emerging platforms. In *International Conference on Parallel Computing Technologies*, pages 153–168. Springer, 2025.

[6] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[7] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.

[8] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.

[9] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4685–4694, 2019.

[10] Emily Denton, Wojciech Zaremba, Joan Bruna, Yann LeCun, and Rob Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 1*, page 1269–1277, Cambridge, MA, USA, 2014. MIT Press.

[11] Moonjung Eo, Suhyun Kang, and Wonjong Rhee. An effective low-rank compression with a joint rank selection followed by a compression-friendly training. *Neural Networks*, 161:165–177, 2023.

[12] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 2943–2952. PMLR, 2020.

[13] Stefan Falkner, Aaron Klein, and Frank Hutter. Combining hyperband and bayesian optimization. In *NIPS 2017 Bayesian Optimization Workshop (Dec 2017)*, 2017.

[14] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. Knowledge distillation: A survey. *Int. J. Comput. Vis.*, 129(6):1789–1819, 2021.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[16] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web*, pages 173–182, 2017.

[17] Yibo He, Cunjian Huang, Xianmiao Qu, Hongdeng Chen, Wei Yang, and Tao Xie. Rvismith: Fuzzing compilers for rvv intrinsics. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, page 768–782, New York, NY, USA, 2025. Association for Computing Machinery.

[18] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[19] Gary B Huang, Marwan Mattar, Tamara Berg, and Eric Learned-Miller. Labeled faces in the wild: A database forstudying face recognition in unconstrained environments. In *Workshop on faces in'Real-Life'Images: detection, alignment, and recognition*, 2008.

[20] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: training neural networks with low precision weights and activations. *J. Mach. Learn. Res.*, 18 (1):6869–6898, 2017.

[21] Yerlan Idelbayev and Miguel A Carreira-Perpinán. Low-rank compression of neural nets: Learning the rank of each layer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8046–8056, 2020.

[22] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018.

[23] Geonhwa Jeong, Sana Damani, Abhimanyu Rajeshkumar Bambhaniya, Eric Qin, Christopher J. Hughes, Sreenivas Subramoney, Hyesoon Kim, and Tushar Krishna. VEGETA: vertically-integrated extensions for sparse/dense GEMM tile acceleration on cpus. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*, pages 259–272. IEEE, 2023.

[24] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News*, 45(1):615–629, 2017.

[25] Zhenhao Li, Wei Hu, and Shuang Chen. Design and implementation of cnn custom processor based on risc-v architecture. In *2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*, pages 1945–1950. IEEE, 2019.

[26] Lucas Liebenwein, Alaa Maalouf, Dan Feldman, and Daniela Rus. Compressing neural networks: Towards determining the optimal layer-wise decomposition. *Advances in Neural Information Processing Systems*, 34:5328–5344, 2021.

[27] Shaohui Lin, Rongrong Ji, Xiaowei Guo, and Xuelong Li. Towards convolutional neural networks compression via global error reconstruction. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, pages 1753–1759, 2016. URL `https://www.ijcai.org/Proceedings/16/Papers/251.pdf`.

[28] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European conference on computer vision*, pages 740–755. Springer, 2014.

[29] Gai Liu, Joseph Primmer, and Zhiru Zhang. Rapid generation of high-quality risc-v processors from functional instruction set specifications. In *Proceedings of the 56th Annual Design Automation Conference 2019*, pages 1–6, 2019.

[30] Jiuyang Liu, Qinjun Li, Yunqian Luo, Hongbin Zhang, Jiongjia Lu, Shupei Fan, Jianhao Ye, Yang Liu, Xiaoyi Liu, Yanqi Yang, et al. Titan-i: An open-source, high performance risc-v vector core. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, pages 675–690, 2025.

[31] Bingqian Lu, Jianyi Yang, Weiwen Jiang, Yiyu Shi, and Shaolei Ren. One proxy device is enough for hardware-aware neural architecture search. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 5(3):1–34, 2021. doi: 10.1145/3491046.

[32] Vincenzo Maisto and Alessandro Cilardo. A pluggable vector unit for risc-v vector extension. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1143–1148. IEEE, 2022.

[33] Julian Oppermann, Brindusa Mihaela Damian-Kosterhon, Florian Meisel, Tammo Mürmann, Eyck Jentzsch, and Andreas Koch. Longnail: High-level synthesis of portable custom instruction set extensions for risc-v processors from descriptions in the open-source coredsl language. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 591–606, 2024.

[34] Angela Pohl, Mirko Greese, Biagio Cosenza, and Ben Juurlink. A performance analysis of vector length agnostic code. In *2019 International Conference on High Performance Computing & Simulation (HPCS)*, pages 159–164. IEEE, 2019.

[35] Angela Pohl, Biagio Cosenza, and Ben Juurlink. Vectorization cost modeling for neon, avx and sve. *Performance Evaluation*, 140:102106, 2020.

[36] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. Relay: A new ir for machine learning frameworks. In *Proceedings of the 2nd ACM SIGPLAN international workshop on machine learning and programming languages*, pages 58–68, 2018.

[37] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.

[38] Anisha Sharma. RISC-V 2024: A year of global growth and innovation. `https://riscv.org/blog/`, December 2024. Accessed: 2026-01-12.

[39] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2017.

[40] Hidenori Tanaka, Daniel Kunin, Daniel L Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. *Advances in neural information processing systems*, 33: 6377–6389, 2020.

[41] Vasileios Titopoulos, Kosmas Alexandridis, Christodoulos Peltekis, Chrysostomos Nicopoulos, and Giorgos Dimitrakopoulos. Indexmac: A custom risc-v vector instruction to accelerate structured-sparse matrix multiplications. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2024.

[42] Chaoqi Wang, Guodong Zhang, and Roger B. Grosse. Picking winning tickets before training by preserving gradient flow. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[43] Jinqi Xiao, Chengming Zhang, Yu Gong, Miao Yin, Yang Sui, Lizhi Xiang, Dingwen Tao, and Bo Yuan. Haloc: hardware-aware automatic low-rank compression for compact neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 10464–10472, 2023.

# A Derivation of Instruction-Profitable Rank Conditions

For a node $s$, let $d_{\text{in}}(s)$ be the reduction dimension of the original matrix multiplication and $d_{\text{out}}(s)$ be its output width. After factorization with rank $r_s$, the original multiplication is replaced by two consecutive multiplications. The first one has reduction dimension $d_{\text{in}}(s)$ and output width $r_s$, while the second one has reduction dimension $r_s$ and output width $d_{\text{out}}(s)$.

**Approximation-based condition.** Recall that a rank is instruction-profitable if

$$\triangle I(s) = I(G'; s) - I(G; s) < 0.$$

Under the approximation

$$I_{\text{tail}} = I_{\text{vec}} + I_{\text{cfg}},$$

let

$$J = I_{\text{vec}} + I_{\text{cfg}}.$$

Then the instruction count for a reduction dimension $C$ can be written as

$$\left\lceil \frac{C}{\eta} \right\rceil J.$$

Therefore, the instruction count before factorization is

$$I(G; s) = d_{\text{out}}(s) \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil J.$$

After factorization, the instruction count becomes

$$I(G'; s) = \left[ r_s \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil + d_{\text{out}}(s) \left\lceil \frac{r_s}{\eta} \right\rceil \right] J.$$

Thus, the instruction-profitable condition

$$I(G'; s) < I(G; s)$$

is equivalent to

$$r_s \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil + d_{\text{out}}(s) \left\lceil \frac{r_s}{\eta} \right\rceil < d_{\text{out}}(s) \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil.$$

Since

$$\left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil > 0,$$

we can rearrange the above inequality as

$$r_s < d_{\text{out}}(s) \left( 1 - \frac{\lceil r_s/\eta \rceil}{\lceil d_{\text{in}}(s)/\eta \rceil} \right).$$

This gives the approximation-based instruction-profitable condition.

**Scaling-based sufficient condition.** The approximation-based condition still contains the piecewise term

$$\left\lceil \frac{r_s}{\eta} \right\rceil.$$

To obtain a simpler, scalable condition, we use the ceiling upper bound

$$\left\lceil \frac{r_s}{\eta} \right\rceil \leq \frac{r_s}{\eta} + 1.$$

Let

$$A_s = \left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil.$$

The approximation-based instruction-profitable condition can be written as

$$r_s A_s + d_{\text{out}}(s) \left\lceil \frac{r_s}{\eta} \right\rceil < d_{\text{out}}(s) A_s.$$

Using the upper bound on the ceiling term, it is sufficient to require

$$r_s A_s + d_{\text{out}}(s)\left(\frac{r_s}{\eta} + 1\right) < d_{\text{out}}(s)A_s.$$

Rearranging gives

$$r_s A_s + \frac{d_{\text{out}}(s)}{\eta} r_s + d_{\text{out}}(s) < d_{\text{out}}(s)A_s,$$

and therefore

$$r_s\left(A_s + \frac{d_{\text{out}}(s)}{\eta}\right) < d_{\text{out}}(s)(A_s - 1).$$

Thus, a sufficient instruction-profitable condition is

$$r_s < \frac{d_{\text{out}}(s)(A_s - 1)}{A_s + \frac{d_{\text{out}}(s)}{\eta}}.$$

Substituting the definition of $A_s$, we obtain

$$r_s < \frac{d_{\text{out}}(s)\left(\left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil - 1\right)}{\left\lceil \frac{d_{\text{in}}(s)}{\eta} \right\rceil + \frac{d_{\text{out}}(s)}{\eta}}.$$

This condition is sufficient because it is derived by replacing $\lceil r_s/\eta \rceil$ with an upper bound. It can be conservative because some ranks that violate this simplified condition may still satisfy the exact approximation-based condition.

## B Alternative Accuracy-Tolerance Formulation

In the main paper, VALOR is formulated as a loss-minimization problem under FLOP-reduction and instruction-profitability constraints. In this appendix, we provide an alternative formulation for deployment scenarios where the user specifies an acceptable accuracy loss and expects the method to maximize efficiency gains under this tolerance.

Given an input set $D$ and an accuracy-loss tolerance $\epsilon$, the goal is to find a transformed graph $G'$ that maximizes instruction-count reduction and FLOP reduction while keeping the predicted accuracy degradation within $\epsilon$:

$$\max_{\{r_s\}_{s\in S}} \quad (\triangle I,\ \triangle F)\,\text{s.t.} \qquad \Delta\mathcal{L}_D(\theta', \theta) \le \epsilon.$$

Here, $\triangle I$ and $\triangle F$ denote the global instruction-count and FLOP changes after restructuring, respectively, while $\Delta\mathcal{L}_D(\theta', \theta) \le \epsilon$ bounds the accuracy degradation. To solve this formulation, VALOR uses the same candidate construction as in the main paper. For each matrix-multiplication node $s \in S$, the viable restructuring choices form a candidate group:

$$D_s = \{0\} \cup \{\, r_s \in \mathbb{N} \mid 1 \le r_s \le \min(d_{\text{in}}(s), d_{\text{out}}(s)),\ \triangle I(s) < 0\},$$

where $r_s = 0$ means that node $s$ is kept unchanged, and $r_s > 0$ specifies the rank used to factorize node $s$. The candidate groups across all nodes define the global strategy space, and a restructuring strategy is represented by

$$\mathbf{r} = \{r_s\}_{s\in S} \in \prod_{s\in S} D_s.$$

Given a rank assignment $\{r_s\}_{s\in S}$, VALOR applies low-rank factorization only to nodes with $r_s > 0$, yielding the transformed graph $G'$ with parameter $\theta'$.

Direct end-to-end evaluation of each $\mathbf{r}$ is prohibitively expensive. VALOR therefore adopts Bayesian Optimization and Hyperband (BOHB) [13] to explore the candidate strategy space, using low-cost proxy evaluations for rapid screening and a small number of end-to-end assessments to confirm the final candidate strategy. During search, candidates whose predicted accuracy degradation $\widehat{\Delta\mathcal{L}}(\mathbf{r})$ exceeds the tolerance $\epsilon$ are filtered out. Among the remaining candidates, VALOR selects the strategy that achieves the best efficiency gain in terms of instruction-count reduction and FLOP reduction.

Table 4: Tolerance-based evaluation under different accuracy-loss tolerance $\epsilon$. For each $\epsilon$, every method selects the most aggressive rank configuration whose accuracy loss does not exceed $\epsilon$. Instruction reduction is averaged over $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$. "–" indicates the method cannot satisfy the tolerance.

| | | **ResNet-50** | | | **MobileNetV2** | | | **ArcFace** | | | **YOLOv4** | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $\epsilon$ | **Method** | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ | A.L.↓ | F.R.↑ | I.R.↑ |
| 0.5% | SVD | – | – | – | – | – | – | 0.41 | 35.9 | 31.7 | – | – | – |
| | ALDS | – | – | – | – | – | – | 0.34 | 38.5 | 34.3 | – | – | – |
| | BSR | – | – | – | – | – | – | 0.49 | 33.2 | 29.1 | – | – | – |
| | LCNN | – | – | – | – | – | – | 0.45 | 34.5 | 30.2 | – | – | – |
| | VALOR | 0.48 | 35.2 | **34.6** | – | – | – | **0.25** | **41.6** | **39.8** | 0.49 | 31.4 | **30.1** |
| 1.0% | SVD | – | – | – | 0.98 | 24.3 | 20.1 | 0.41 | 35.9 | 31.7 | 0.94 | 25.6 | 21.3 |
| | ALDS | – | – | – | 0.92 | 27.1 | 23.5 | 0.34 | 38.5 | 34.3 | 0.89 | 28.4 | 24.8 |
| | BSR | – | – | – | – | – | – | 0.49 | 33.2 | 29.1 | 0.97 | 23.8 | 19.2 |
| | LCNN | – | – | – | – | – | – | 0.45 | 34.5 | 30.2 | 0.93 | 24.9 | 20.5 |
| | VALOR | **0.75** | **43.1** | **41.8** | **0.96** | **36.8** | **35.3** | **0.25** | **41.6** | **39.8** | **0.97** | **39.7** | **37.4** |
| 1.5% | SVD | 1.31 | 37.6 | 33.1 | 1.47 | 29.8 | 25.6 | 0.41 | 35.9 | 31.7 | 1.42 | 31.5 | 27.2 |
| | ALDS | 1.04 | 40.3 | 36.2 | 1.47 | 34.5 | 30.9 | 0.34 | 38.5 | 34.3 | 1.44 | 36.4 | 32.7 |
| | BSR | 1.43 | 33.5 | 29.2 | 1.48 | 26.7 | 22.4 | 0.49 | 33.2 | 29.1 | 1.46 | 28.3 | 23.9 |
| | LCNN | 1.48 | 36.1 | 31.6 | 1.49 | 28.4 | 24.0 | 0.45 | 34.5 | 30.2 | 1.47 | 30.1 | 25.8 |
| | VALOR | **0.98** | **46.5** | **45.1** | **1.23** | **39.4** | **38.2** | **0.31** | **43.8** | **42.1** | **1.18** | **42.9** | **40.8** |
| 2.0% | SVD | 1.82 | 41.3 | 36.8 | 1.95 | 31.7 | 27.4 | 0.41 | 35.9 | 31.7 | 1.87 | 33.8 | 29.5 |
| | ALDS | 1.56 | 44.1 | 39.7 | 1.78 | 37.2 | 33.5 | 0.34 | 38.5 | 34.3 | 1.72 | 39.1 | 35.2 |
| | BSR | 1.62 | 34.8 | 30.5 | 2.00 | 28.9 | 24.6 | 0.49 | 33.2 | 29.1 | 1.94 | 30.2 | 25.7 |
| | LCNN | 1.73 | 38.6 | 34.0 | 1.96 | 30.8 | 26.3 | 0.45 | 34.5 | 30.2 | 1.89 | 32.0 | 27.5 |
| | VALOR | **1.24** | **49.8** | **48.3** | **1.52** | **42.1** | **40.7** | **0.37** | **45.3** | **43.6** | **1.43** | **45.6** | **43.5** |

A.L. = Accuracy Loss (%); F.R. = FLOPs Reduction (%); I.R. = Instruction Reduction (%); $\epsilon$ = accuracy-loss ceiling. "–" = method cannot achieve any compression within the tolerance.

### B.1 Experimental Results under Accuracy Tolerance

Table 4 reports the results under different accuracy-loss tolerance $\epsilon$. For each $\epsilon$, every method selects the most aggressive rank configuration whose accuracy loss does not exceed the specified bound, and instruction reduction is averaged over $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$. Under strict tolerances, several baselines cannot find a compressed configuration for some models, while VALOR still obtains feasible solutions on more tasks. For example, when $\epsilon = 1.0\%$, SVD, ALDS, BSR, and LCNN fail to produce a feasible compressed ResNet-50 model, whereas VALOR achieves $43.1\%$ FLOP reduction and $41.8\%$ instruction reduction with $0.75\%$ accuracy loss.

When the tolerance is relaxed, VALOR consistently achieves stronger efficiency gains. At $\epsilon = 1.5\%$, VALOR improves the average instruction reduction to $41.6\%$, compared with $33.5\%$ for ALDS, the strongest baseline on average. At $\epsilon = 2.0\%$, VALOR further increases the average instruction reduction to $44.0\%$, while ALDS reaches $35.7\%$. VALOR also provides higher FLOP reduction across all four models under both tolerances. These results show that the advantage of RVV-aware rank selection is not tied to the main loss-minimization formulation; it also holds when the user specifies an explicit accuracy-loss tolerance and asks for the most aggressive efficient restructuring within that tolerance.

## C Derivation and Estimation of the Hessian-Guided Ranking Proxy

This appendix provides the derivation and implementation details of the Hessian-guided ranking proxy used in Section 3.3. The goal is not to exactly predict the final validation accuracy, but to obtain a low-cost ordering of candidate rank assignments before expensive end-to-end evaluation. The proxy is used to compare candidate restructuring strategies during search.

**Parameter perturbation induced by low-rank restructuring.** Given a rank assignment $\mathbf{r} = \{r_s\}_{s\in S}$, let

$$S_{\mathbf{r}} = \{s \in S \mid r_s > 0\}$$

denote the set of transformed nodes. For each transformed node $s \in S_{\mathbf{r}}$, VALOR replaces the original weight matrix $W_s$ with the product $W_u W_r$, where $W_u$ and $W_r$ are obtained from the truncated SVD of $W_s$ under the selected rank $r_s$. This replacement induces a parameter perturbation

$$\Delta W_s := W_u W_r - W_s.$$

For nodes that are not transformed, we set

$$\Delta W_s := \mathbf{0}, \qquad s \notin S_{\mathbf{r}}.$$

Thus, a global rank assignment $\mathbf{r}$ induces a structured perturbation $\Delta\theta = \{\Delta W_s\}_{s\in S}$ to the original parameter set $\theta = \{W_s\}_{s\in S}$.

**Second-order approximation of the loss change.** Let $\mathcal{L}_D(\theta)$ denote the loss evaluated on the input set $D$. For each node, define vectorized coordinates $w_s = \mathrm{vec}(W_s)$ and $\delta_s = \mathrm{vec}(\Delta W_s)$. Let $H_{ts} = \partial^2 \mathcal{L}_D / (\partial w_t \partial w_s^\top)$, whose shape is $\dim(w_t) \times \dim(w_s)$. Applying a second-order Taylor expansion around the original parameters gives the dimensionally valid expression

$$\Delta\mathcal{L}_D = \mathcal{L}_D(\theta + \Delta\theta) - \mathcal{L}_D(\theta) \approx \sum_{s\in S} \nabla_{w_s} \mathcal{L}_D^\top \delta_s + \frac{1}{2} \sum_{s,t\in S} \delta_t^\top H_{ts} \delta_s,$$

where rectangular cross-layer blocks are multiplied only in vectorized parameter coordinates. Since the starting model is already trained, the first-order term is typically small for nearby low-rank perturbations. The quadratic term motivates the curvature and coupling scores used by VALOR:

$$\Delta\mathcal{L}_D \approx \frac{1}{2} \sum_{s,t\in S} \delta_t^\top H_{ts} \delta_s.$$

This expansion motivates the proxy but is not evaluated faithfully: VALOR normalizes the local distortion and curvature signals and folds pairwise interactions into an additive score. The implemented score is therefore an empirical ranking heuristic, not an exact second-order loss predictor.

**Local and cross-layer contributions.** The quadratic approximation can be separated into diagonal and off-diagonal parts:

$$\Delta\mathcal{L}_D \approx \frac{1}{2} \sum_{s\in S} \delta_s^\top H_{ss} \delta_s + \frac{1}{2} \sum_{\substack{s,t\in S \\ s\neq t}} \delta_t^\top H_{ts} \delta_s.$$

The diagonal term measures the sensitivity of the loss to the perturbation introduced at a single node. The off-diagonal term measures the interaction between perturbations introduced at two different nodes. Directly evaluating all diagonal and off-diagonal quadratic terms for every candidate $\mathbf{r}$ would be expensive and would couple all rank decisions together. VALOR therefore keeps a decomposable local error term and uses a normalized coupling score to account for the dominant cross-layer effect.

**Local factorization error.** For each transformed node $s \in S_{\mathbf{r}}$, VALOR measures the local distortion caused by the selected rank $r_s$ using the normalized factorization error

$$e_s(r_s) = \ell_s = \frac{\|W_s - W_u W_r\|_F^2}{\|W_s\|_F^2} = \frac{\|\Delta W_s\|_F^2}{\|W_s\|_F^2}.$$

This quantity is cheap to compute once $W_u$ and $W_r$ are generated by truncated SVD. It measures the relative magnitude of the local perturbation:

$$\|\Delta W_s\|_F = \sqrt{\ell_s}\,\|W_s\|_F.$$

Because $\|\delta_s\|_2 = \|\Delta W_s\|_F$, the diagonal quadratic term satisfies

$$\left|\delta_s^\top H_{ss}\delta_s\right| \le \|H_{ss}\|_F \cdot \|\delta_s\|_2^2.$$

Therefore, the local contribution grows with both the perturbation magnitude $\|\Delta W_s\|_F^2$ and the local curvature magnitude $\|H_{ss}\|_F$. In the ranking proxy, $e_s(r_s)$ captures the rank-dependent part of this local distortion, while the coupling-aware coefficient introduced below accounts for the relative sensitivity and interaction among transformed nodes.

**Bounding the cross-layer coupling.** For two transformed nodes $t$ and $s$, the cross-layer quadratic term is

$$c_{ts} = \frac{1}{2}\delta_t^\top H_{ts}\delta_s.$$

Using the induced bilinear bound, its magnitude is bounded by

$$|c_{ts}| \le \frac{1}{2}\|H_{ts}\|_F\|\delta_t\|_2\|\delta_s\|_2.$$

This bound shows that the interaction between two transformed nodes depends on two factors: the magnitude of the perturbations and the curvature coupling between the corresponding parameter blocks. To make the coupling strength comparable across layers, VALOR normalizes the off-diagonal Hessian block by the diagonal block magnitudes:

$$g_{ts} = \frac{\|H_{ts}\|_F}{\sqrt{\|H_{tt}\|_F \cdot \|H_{ss}\|_F}}.$$

The score $g_{ts}$ is dimensionless. A larger $g_{ts}$ indicates that perturbations at nodes $t$ and $s$ are more strongly coupled by the loss curvature. In practice, $g_{ts}$ is clipped to $[0, 1]$ to avoid unstable values caused by estimation noise.

**From pairwise coupling to an additive ranking proxy.** The exact quadratic approximation contains pairwise terms for all transformed node pairs. This is inconvenient for BOHB search because the score of one rank choice would depend on every other rank choice through dense pairwise terms. VALOR therefore uses an additive proxy that preserves node-wise decomposability while still reflecting the dominant coupling effect.

Let $t \prec s$ denote that node $t$ precedes node $s$ in the topological order of the computation graph. Perturbations introduced at earlier nodes can propagate through the graph and affect later transformed nodes. VALOR accumulates this preceding influence into an amplification coefficient:

$$\alpha_s = 1 + \sum_{t \prec s,\ t \in S_{\mathbf{r}}} \ell_t \cdot g_{ts}.$$

Here, $\ell_t$ measures the local distortion introduced at the preceding node $t$, and $g_{ts}$ measures how strongly the perturbation at $t$ is coupled with node $s$ through the loss curvature. The constant 1 keeps the local error of node $s$ even when there is no preceding transformed node. Since $\ell_t \ge 0$ and $g_{ts} \ge 0$, we have $\alpha_s \ge 1$.

Finally, VALOR ranks a global rank assignment $\mathbf{r}$ by

$$\widehat{\Delta\mathcal{L}}(\mathbf{r}) = \sum_{s \in S_{\mathbf{r}}} \alpha_s \cdot e_s(r_s).$$

This proxy has three useful properties. First, it is cheap to evaluate after the candidate ranks and local factorization errors are computed. Second, it penalizes aggressive rank choices through $e_s(r_s)$. Third, it penalizes rank choices at nodes that are affected by strongly coupled preceding perturbations through $\alpha_s$. Thus, it provides a practical ordering of candidate restructuring strategies and reduces the number of candidates that require held-out full evaluation.

**Estimating Hessian block norms.** VALOR does not explicitly construct the full Hessian $H$. Instead, it estimates the block norms needed for $g_{ts}$ on a small subset of $D$ using a block-wise empirical Fisher approximation. Let $D_{\mathrm{cal}}$ denote this calibration subset. For each sample $x \in D_{\mathrm{cal}}$, we compute the gradient of the loss with respect to the parameters of node $s$, denoted by

$$G_s(x) = \nabla_{W_s} \mathcal{L}(x; \theta), \qquad g_s(x) = \mathrm{vec}(G_s(x)).$$

The diagonal block magnitude is estimated from gradient outer products:

$$\|H_{ss}\|_F \approx \left\| \frac{1}{|D_{\mathrm{cal}}|} \sum_{x \in D_{\mathrm{cal}}} g_s(x) g_s(x)^\top \right\|_F .$$

Similarly, the off-diagonal coupling magnitude between nodes $t$ and $s$ is estimated by

$$\|H_{ts}\|_F \approx \left\| \frac{1}{|D_{\mathrm{cal}}|} \sum_{x \in D_{\mathrm{cal}}} g_t(x) g_s(x)^\top \right\|_F .$$

Substituting these estimates into the definition of $g_{ts}$ gives the coupling score used in the proxy. This avoids forming the dense Hessian and only requires gradients for the transformed matrix-multiplication nodes on a small calibration subset. Therefore, the proxy remains lightweight enough to be used inside the BOHB search loop.

**Remarks on approximation accuracy.** The ranking proxy is designed to compare candidate rank assignments, not to serve as an exact predictor of the final task metric. Several approximations are involved: the first-order term is ignored because the original model is already trained; absolute diagonal curvature scales are normalized rather than retained; the Hessian is approximated by empirical Fisher block norms; and pairwise coupling terms are folded into the additive amplification coefficient $\alpha_s$. Consequently, the proxy should not be interpreted as a faithful second-order loss approximation. We evaluate it empirically by comparing its ranking quality and selected configurations with normalized SVD error and a diagonal-curvature alternative in Appendix E.4.

# D Per-Configuration Results Across Effective Vector Lengths

Figure 2 reports the full per-$\eta$ results for all models and methods. The results complement the averaged results in Table 1 by showing how accuracy loss, FLOP reduction, and instruction reduction change under different RVV effective vector lengths.

A clear difference can be observed between VALOR and the hardware-agnostic baselines. For SVD, ALDS, BSR, and LCNN, the selected ranks are fixed across RVV configurations. Therefore, their accuracy loss and FLOP reduction remain constant for all $\eta$, while their instruction reduction changes only because the same transformed model is evaluated under different vector lengths. In contrast, VALOR selects ranks separately for each $\eta$, so all three metrics can vary across configurations.

Across the seven effective vector lengths, VALOR consistently achieves the strongest instruction reduction. For example, on ResNet-50, VALOR improves instruction reduction from $34.8\%$ at $\eta = 4$ to $47.3\%$ at $\eta = 256$, while ALDS ranges from $30.4\%$ to $39.9\%$. Similar trends are observed on MobileNetV2, ArcFace, and YOLOv4, where VALOR reaches $40.8\%$, $45.3\%$, and $42.9\%$ instruction reduction at $\eta = 256$, respectively. These results indicate that profile-aware rank selection enables VALOR to exploit larger effective vector lengths more effectively than fixed-rank baselines.

VALOR also improves FLOP reduction as $\eta$ increases. This is because larger effective vector lengths enlarge the set of instruction-profitable rank choices, giving VALOR more freedom to select low-loss and high-compression restructuring strategies. For instance, VALOR's FLOP reduction increases from $39.1\%$ to $46.3\%$ on ResNet-50 and from $35.7\%$ to $42.9\%$ on YOLOv4 as $\eta$ increases from $4$ to $256$. At the same time, the accuracy loss decreases under larger $\eta$, dropping from $1.05$ to $0.47$ on ResNet-50 and from $1.27$ to $0.69$ on YOLOv4. This shows that the instruction-profitable condition does not simply force aggressive rank reduction. Instead, it changes the feasible rank space according to the target RVV profile and allows VALOR to find better accuracy-efficiency trade-offs.

The baselines show less stable instruction behavior across configurations. Although their accuracy loss and FLOP reduction remain unchanged, their instruction reduction can change non-monotonically

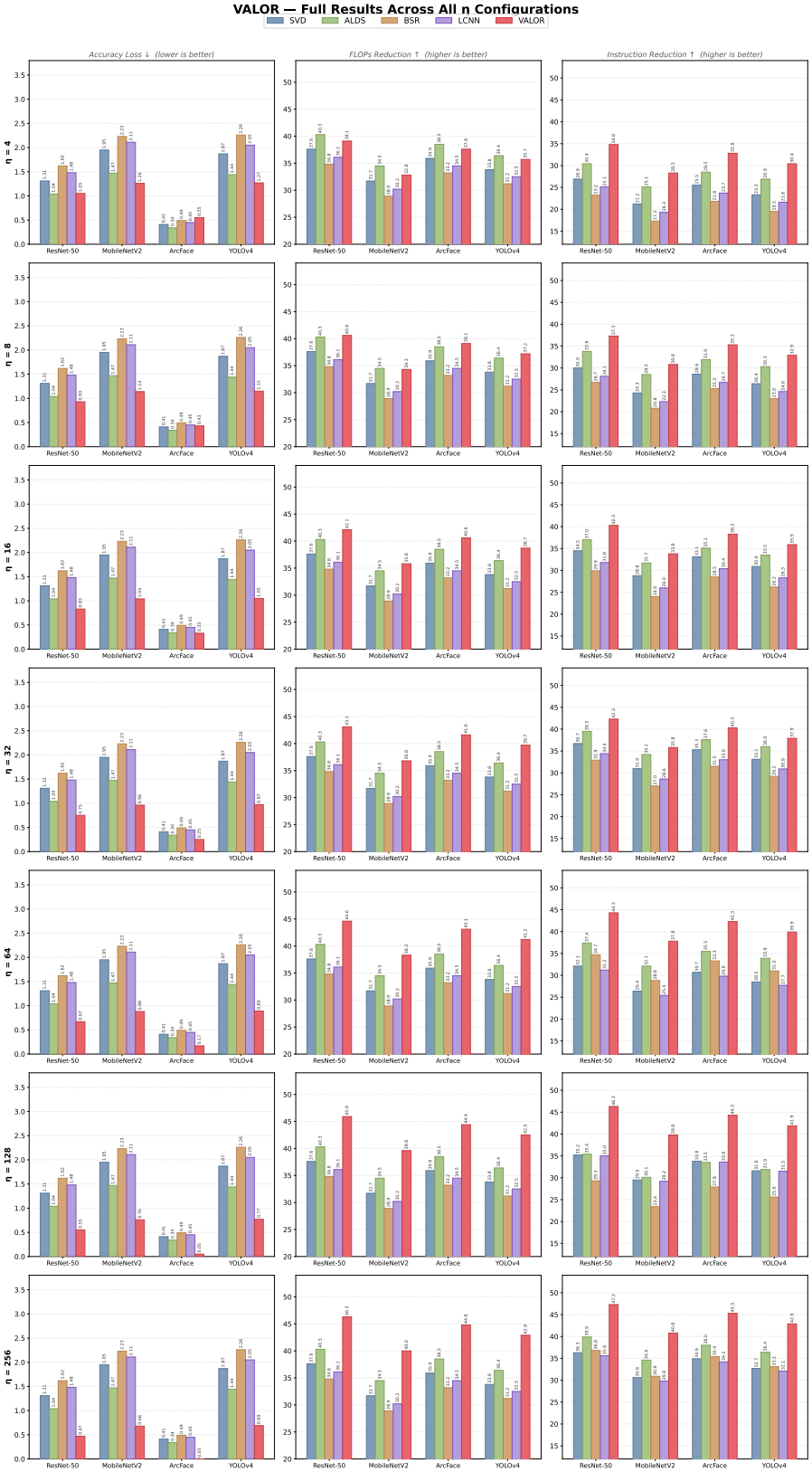



Figure 2: Per-configuration results across RVV effective vector lengths. For each $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$, we report accuracy loss, FLOP reduction, and instruction reduction for SVD, ALDS, BSR, and VALOR on four models.

as $\eta$ varies. This occurs because fixed ranks interact differently with vector chunk boundaries under different effective vector lengths. For example, ALDS on ResNet-50 achieves $39.5\%$ instruction reduction at $\eta = 32$, decreases to $35.4\%$ at $\eta = 128$, and then increases to $39.9\%$ at $\eta = 256$. A similar pattern appears for LCNN on ResNet-50, where instruction reduction decreases from $34.4\%$ at $\eta = 32$ to $31.2\%$ at $\eta = 64$, then increases to $35.6\%$ at $\eta = 256$. Such variation confirms that FLOP-oriented or hardware-agnostic rank selection alone cannot reliably predict RVV instruction efficiency.

Overall, the per-configuration results validate the motivation for RVV-aware restructuring. A single hardware-agnostic rank assignment can produce unstable instruction-level benefits across vector profiles, while VALOR adapts the rank choices to each $\eta$ and consistently improves instruction reduction with lower average accuracy loss.

# E Additional Validation of Modeling and Search

This appendix provides additional validation for the modeling assumptions and search procedure used by VALOR. We first evaluate the tail-cost approximation with controlled microbenchmarks on Spike. We then report the BOHB search budget used in our experiments. Finally, we evaluate the ranking quality of the Hessian-guided proxy and its sensitivity to the calibration set size.

## E.1 Tail-Cost Approximation Microbenchmark

VALOR uses the approximation $I_{\text{tail}} = I_{\text{vec}} + I_{\text{cfg}}$ to model the cost of a tail iteration. To validate this approximation, we construct controlled matrix-multiplication microbenchmarks with fixed $B = 128$ and $D = 512$, and vary the reduction dimension $C$ around each effective vector length $\eta$. For each $\eta$, we compare the aligned case $C = \eta$ and the near-tail case $C = \eta + 1$. Table 5 reports the predicted instruction count and the measured dynamic instruction count on Spike.

Table 5: Microbenchmark validation of the tail-cost approximation ($B = 128$, $D = 512$, measured on Spike).

| $\eta$ | $C$ | Pred. Inst. | Meas. Inst. | Error |
|---|---|---|---|---|
| 4 | 4 | 3 072 | 3 088 | 0.52% |
| 4 | 5 | 3 840 | 3 901 | 1.56% |
| 8 | 8 | 3 072 | 3 094 | 0.71% |
| 8 | 9 | 3 456 | 3 518 | 1.76% |
| 16 | 16 | 3 072 | 3 101 | 0.93% |
| 16 | 17 | 3 264 | 3 327 | 1.89% |
| 32 | 32 | 3 072 | 3 108 | 1.16% |
| 32 | 33 | 3 168 | 3 237 | 2.13% |
| 64 | 64 | 3 072 | 3 115 | 1.38% |
| 64 | 65 | 3 120 | 3 189 | 2.16% |
| 128 | 128 | 3 072 | 3 126 | 1.73% |
| 128 | 129 | 3 096 | 3 165 | 2.18% |
| 256 | 256 | 3 072 | 3 139 | 2.13% |
| 256 | 257 | 3 084 | 3 158 | 2.34% |

Pred. Inst. computed using the instruction-count model with $I_{\text{tail}} = I_{\text{vec}} + I_{\text{cfg}}$; Meas. Inst. from Spike dynamic instruction count.

The measured instruction counts closely follow the predicted counts. Across all tested cases, the prediction error remains below $2.34\%$. The aligned cases $C = \eta$ consistently avoid the additional tail iteration, while the near-tail cases $C = \eta + 1$ introduce the expected tail overhead. This confirms that the approximation captures the dominant instruction-count change caused by tail handling.

## E.2 BOHB Search Budget

Table 6 reports the search budget used by VALOR. For each model and effective vector length, VALOR first constructs candidate rank groups using the instruction-profitable condition, then uses BOHB to score candidates with the Hessian-guided proxy and promote a small number of candidates for full validation. The reported values are averaged over $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$.

Table 6: Complete BOHB search budget used by VALOR (averaged over $\eta \in \{4, 8, 16, 32, 64, 128, 256\}$).

| **Model** | Avg. $\lvert D_s \rvert$ | Proxy evals | Full evals | Search time |
|---|---|---|---|---|
| ResNet-50 | 1 024 | 384 | 24 | 47 min |
| MobileNetV2 | 576 | 288 | 18 | 23 min |
| ArcFace | 832 | 336 | 21 | 38 min |
| YOLOv4 | 1 280 | 432 | 27 | 62 min |

$\lvert D_s \rvert$ = average candidate group size per $\eta$; Proxy evals scored by Hessian-guided proxy; Full evals validated on held-out set. Search time includes candidate construction, proxy ranking, promotion, and held-out validation, measured with a single NVIDIA RTX A6000.

Table 7: Budget-matched search comparison. Each entry reports accuracy loss (percentage points) / X60 latency reduction (%).

| **Model** | **BOHB** | **Random search** |
|---|---|---|
| ResNet-50 | **0.75 / 38.2** | 0.93 / 36.4 |
| MobileNetV2 | **0.96 / 31.4** | 1.18 / 29.5 |
| ArcFace | **0.25 / 35.1** | 0.33 / 33.3 |
| YOLOv4 | **0.97 / 33.0** | 1.17 / 31.0 |

The number of full validations is much smaller than the number of proxy-scored candidates. Thus, the proxy reduces, but does not eliminate, held-out full evaluations during BOHB search. The 23–62 minute totals here cover the complete search and are distinct from the 140–300 second candidate-ranking-stage times in Table 3.

**Budget-matched search baseline.** We choose BOHB because the search contains many inexpensive proxy evaluations but permits only a limited number of costly held-out validations. Its successive-halving schedule allocates more validation budget to promising candidates; it does not provide a global-optimality guarantee. To test whether the conclusions depend on this choice, we compare BOHB with uniform random search over the same instruction-profitable rank space. Both methods use the same proposal and validation budgets, fine-tuning protocol, and stopping rule.

Under the matched budget, BOHB reduces average accuracy loss by 0.17 percentage points and improves average latency reduction by 1.9 percentage points. This supports BOHB as a practical budget-aware optimizer for the evaluated search space without implying global optimality.

### E.3 Ranking Quality of the Hessian-Guided Proxy

To evaluate whether the Hessian-guided proxy provides a reliable ordering of candidate rank assignments, we randomly sample $m = 100$ candidate strategies for each model and compare the predicted loss increase $\widehat{\Delta\mathcal{L}}$ with the measured validation loss increase before final model selection. We report Spearman's rank correlation $\rho$ and Kendall's rank correlation $\tau$ in Table 8.

The proxy achieves strong rank correlation with the measured validation loss increase across all models, with an average Spearman correlation of $0.929$ and an average Kendall correlation of $0.808$. This indicates that the proxy is effective for ordering candidate rank assignments during search, even though it is not intended to be an exact predictor of the final task metric.

### E.4 Comparison with Simpler Ranking Proxies

We further compare normalized SVD error, a diagonal-curvature-weighted local score, and the coupling-aware proxy on the same 100 candidates per model and effective vector length. Measured validation-loss increases provide the ranking ground truth, and the instruction-profitable filter is fixed across all three variants.

Diagonal curvature improves both ranking quality and selected accuracy over normalized SVD error. Adding the normalized coupling signal provides a further 0.18 percentage-point reduction in average

Table 8: Ranking quality of the Hessian-guided proxy ($m = 100$ sampled candidates per model, averaged over all $\eta$).

| **Model** | Spearman $\rho$ | Kendall $\tau$ |
|---|---|---|
| ResNet-50 | 0.936 | 0.814 |
| MobileNetV2 | 0.912 | 0.783 |
| ArcFace | 0.948 | 0.837 |
| YOLOv4 | 0.921 | 0.796 |
| *Average* | *0.929* | *0.808* |

Table 9: Ranking quality and selected-configuration quality for three proxies on identical candidates. A.L. is in percentage points; F.R. and I.R. are in percent.

| **Proxy** | **Spearman $\rho$** | **Kendall $\tau$** | **Avg. A.L./F.R./I.R.** |
|---|---|---|---|
| Normalized SVD | 0.602 | 0.435 | 1.06 / 39.5 / 37.7 |
| Diagonal curvature | 0.762 | 0.583 | 0.91 / 40.0 / 38.3 |
| Coupling-aware | **0.929** | **0.808** | **0.73 / 40.3 / 38.8** |

accuracy loss and slightly improves the two efficiency metrics. These results empirically support the heuristic signals used by the proxy; they do not turn it into an exact second-order loss approximation.

### E.5 Candidates Rejected by the Constraints

To check whether the two constraints exclude better latency points, we sample candidates from the broader valid-rank pool for ResNet-50 and YOLOv4 at $\eta = 32$. Feasible candidates (F) satisfy both constraints; FV violates only the FLOP constraint; IV satisfies the FLOP constraint but contains at least one node that violates the instruction-profitability condition. All groups use the same optimizer, budget, seeds, and ranking proxy. Table values are group means, and the VALOR-selected configuration is included in F.

The sampled FV candidates preserve accuracy but increase FLOPs and provide little latency reduction. The sampled IV candidates reduce FLOPs but increase predicted RVV instructions and have negative mean latency reduction. The best sampled FV and IV candidates also remain below the best feasible candidate in X60 latency reduction for both models. This diagnostic does not prove that no excluded point can ever be beneficial, but it finds no better excluded point in the evaluated sample.

### E.6 Sensitivity to Calibration Set Size

The Hessian-guided proxy estimates block-wise curvature information from a calibration subset. To assess the sensitivity to the calibration set size, we vary the number of calibration samples and report the resulting Spearman correlation in Table 11.

The ranking quality improves as the calibration set grows from 64 to 256 samples and then saturates. The gain from 256 to 512 samples is small across all models. Therefore, we use 256 calibration samples in the main experiments to balance ranking quality and proxy-evaluation cost.

## F Compute Resources

All experiments are conducted on a machine running Ubuntu 22.04 (64-bit) with an Intel Core i9 CPU at 2.10 GHz, 120 GB RAM, and one NVIDIA RTX A6000 GPU. The GPU is mainly used for fine-tuning and accuracy evaluation, while the RVV instruction-count analysis, rank-candidate filtering, and BOHB search are executed on CPU. The real-device latency experiments are conducted on a cloud-hosted SpacemiT X60 RISC-V vector processor. The 140–300 s values in Table 3 isolate the candidate-ranking stage. The complete BOHB search, including candidate construction, proxy ranking, promotion, and held-out validation, takes 23–62 minutes per model as reported in Table 6; both measurements use the RTX A6000 environment above. Including low-rank factorization, fine-tuning, baseline evaluation, Spike-based RVV profiling, and real-device latency measurement, the

Table 10: Rejected-candidate analysis. $\Delta F$ and $\Delta I$ are changes relative to the original model; positive latency values denote reductions.

| **Model** | **Group** | **A.L.** | $\Delta F$ | $\Delta I$ | **Lat. red.** |
|---|---|---|---|---|---|
| ResNet-50 | F | 1.52 | $-37.4\%$ | $-35.8\%$ | 32.1% |
| | FV | 0.23 | $+2.3\%$ | $-3.5\%$ | 3.6% |
| | IV | 1.38 | $-26.7\%$ | $+3.2\%$ | $-1.4\%$ |
| YOLOv4 | F | 1.74 | $-35.8\%$ | $-33.6\%$ | 29.5% |
| | FV | 0.21 | $+2.6\%$ | $-3.0\%$ | 3.2% |
| | IV | 1.51 | $-24.9\%$ | $+3.8\%$ | $-1.8\%$ |

Table 11: Sensitivity of proxy ranking quality (Spearman $\rho$) to calibration set size.

| **Model** | 64 | 128 | 256 | 512 |
|---|---|---|---|---|
| ResNet-50 | 0.891 | 0.924 | 0.936 | 0.940 |
| MobileNetV2 | 0.867 | 0.898 | 0.912 | 0.917 |
| ArcFace | 0.904 | 0.933 | 0.948 | 0.951 |
| YOLOv4 | 0.879 | 0.908 | 0.921 | 0.925 |

Default calibration size used in all other experiments: 256.

full experimental pipeline takes approximately 2–4 GPU-days and 5–8 CPU-days in total. The SpacemiT X60 evaluation takes approximately 10–20 device-hours, depending on queueing overhead and repeated latency measurements on the cloud platform.

## NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the main contribution of VALOR as an RVV-aware low-rank restructuring framework for efficient RISC-V inference. The claims are supported by the methodology, the instruction-profitability analysis, and the experimental results reported in Sections 3 and 4.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper includes a limitation discussion in Section 6. It discusses the dependence on a fixed TVM-based compilation pipeline and the possible effects of backend code generation, operator fusion, memory layout, and vectorization schedules.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The paper states the assumptions used for the RVV instruction-count approximation and provides the derivation of the instruction-profitable rank conditions in Appendix A. The Hessian-guided ranking proxy and its estimation procedure are further derived in Appendix C.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper describes the evaluated models, datasets, RVV configurations, baseline methods, metrics, and the TVM-based deployment pipeline in Section 4.1. The implementation is also released through an anonymized repository to support reproduction of the main experimental results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The paper provides an anonymized open-source implementation of VALOR. The repository includes the code and instructions for reproducing the main experiments, while the evaluated datasets are publicly available through their original sources.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The experimental setup in Section 4.1 specifies the model–dataset pairs, RVV configurations, baselines, evaluation metrics, deployment pipeline, and fine-tuning protocol. Additional details for rank search and evaluation are provided in the appendices and the released implementation.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper reports the mean results over three runs, but it does not currently include error bars, confidence intervals, or statistical significance tests. We include the number of runs to indicate the source of averaging, but a more detailed statistical analysis is left for future extensions.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper reports the main compute environment, including the Ubuntu version, CPU, memory, GPU, and the cloud-hosted SpacemiT X60 platform used for real-device latency evaluation. It also reports the BOHB search time and provides an estimated total compute budget for the full experimental pipeline.

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research uses standard public datasets, public model architectures, and standard compiler and hardware evaluation workflows. We are not aware of any aspect of the work that violates the NeurIPS Code of Ethics.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The work can have positive impact by improving the efficiency and accessibility of edge AI deployment on open RISC-V platforms. Potential negative impacts are limited because the method is a general inference-efficiency technique and does not introduce new high-risk datasets, models, or data-generation capabilities.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release high-risk pretrained models, image generators, language models, or scraped datasets. The released asset is an implementation of an RVV-aware model-restructuring method.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper cites the original works for the datasets, models, compiler infrastructure, and baseline methods used in the evaluation.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

Justification: The paper introduces and releases the VALOR implementation as a new software asset through an anonymized repository. Documentation and reproduction instructions are provided alongside the released code.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing experiments or research with human subjects. Therefore, participant instructions, screenshots, and compensation details are not applicable.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or human-subject research. Therefore, IRB approval or an equivalent review is not applicable.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: The core method development, experiments, and scientific claims do not rely on LLMs as an important, original, or non-standard component. LLMs were used only for writing, editing, or formatting assistance, which does not affect the methodology, results, or originality of the research.